

Large Language Models: From N-Grams to the Attention Revolution

BINF 4002 — Machine Learning for Health

Matthew McDermott

Department of Biomedical Informatics, Columbia University

1. **The Paradox** — Searle's Chinese Room
2. **Language Modeling** — what the problem really is
3. **Why It's Hard** — sparsity, n-grams, combinatorics
4. **The Transformer** — how attention solves it
5. **The LLM Ecosystem** — how to actually use them
6. **Open Challenges** — what remains unsolved

Experiments and walkthroughs in the companion notebook.

❖ **Notebook:** `llm_lecture_notebook.ipynb`

The Paradox

Today's Paradox: The Chinese Room

Searle (1980): A thought experiment

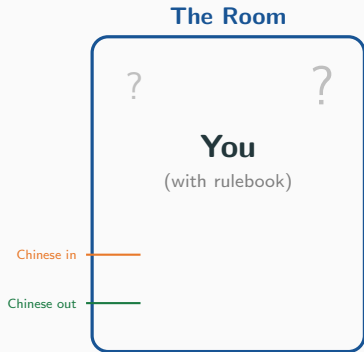
Imagine you are locked in a room.

Today's Paradox: The Chinese Room

Searle (1980): A thought experiment

Imagine you are locked in a room.

- You receive slips of paper with **Chinese symbols**.

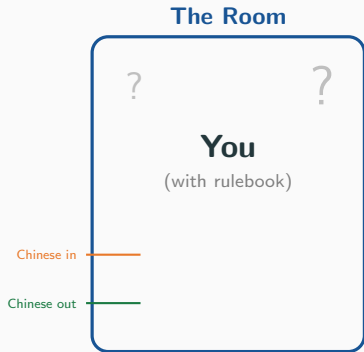


Today's Paradox: The Chinese Room

Searle (1980): A thought experiment

Imagine you are locked in a room.

- You receive slips of paper with **Chinese symbols**.
- A **rulebook** (in English) tells you exactly which symbols to write back.

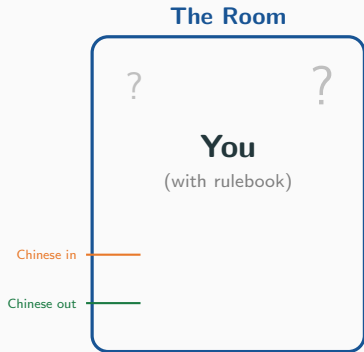


Today's Paradox: The Chinese Room

Searle (1980): A thought experiment

Imagine you are locked in a room.

- You receive slips of paper with **Chinese symbols**.
- A **rulebook** (in English) tells you exactly which symbols to write back.
- You pass responses through a slot.

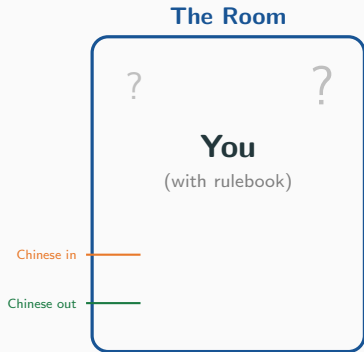


Today's Paradox: The Chinese Room

Searle (1980): A thought experiment

Imagine you are locked in a room.

- You receive slips of paper with **Chinese symbols**.
- A **rulebook** (in English) tells you exactly which symbols to write back.
- You pass responses through a slot.
- To anyone outside, you appear to **understand Chinese**.



Today's Paradox: The Chinese Room

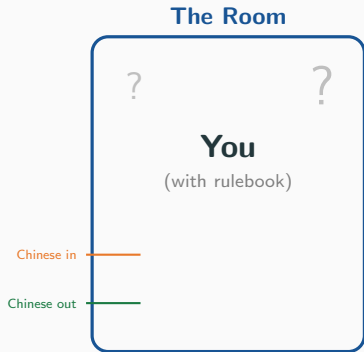
Searle (1980): A thought experiment

Imagine you are locked in a room.

- You receive slips of paper with **Chinese symbols**.
- A **rulebook** (in English) tells you exactly which symbols to write back.
- You pass responses through a slot.
- To anyone outside, you appear to **understand Chinese**.

The question

You follow rules. You produce fluent output.
But do you *understand* anything?



Why Philosophers Cared — A Lot

The target: Strong AI

The dominant view before 1980: **functionalism**.
If a system produces the right outputs from the right inputs, it *has* a mind. Behavior is sufficient for understanding.

Why Philosophers Cared — A Lot

The target: Strong AI

The dominant view before 1980: **functionalism**.
If a system produces the right outputs from the right inputs, it *has* a mind. Behavior is sufficient for understanding.

Searle's challenge

The Chinese Room shows you can produce **all the right outputs** with **zero understanding**.
Syntax alone cannot generate semantics.

Why Philosophers Cared — A Lot

The target: Strong AI

The dominant view before 1980: **functionalism**. If a system produces the right outputs from the right inputs, it *has* a mind. Behavior is sufficient for understanding.

Searle's challenge

The Chinese Room shows you can produce **all the right outputs** with **zero understanding**. Syntax alone cannot generate semantics.

Why the field split: replies

- **Systems reply:** the whole *system* understands, not just you.
Searle: memorize the full rulebook — do you understand now?
- **Robot reply:** ground the system in a body and sensors.
Searle: still symbol manipulation, just with richer inputs.
- **Brain simulator:** simulate actual neurons.
Searle: simulating water doesn't make you wet.
- **Functionalists:** behavior *is* mind. None of these replies concede the point.

Why Philosophers Cared — A Lot

The target: Strong AI

The dominant view before 1980: **functionalism**. If a system produces the right outputs from the right inputs, it *has* a mind. Behavior is sufficient for understanding.

Searle's challenge

The Chinese Room shows you can produce **all the right outputs** with **zero understanding**. Syntax alone cannot generate semantics.

The verdict

One of the most cited papers in philosophy of mind. No consensus. The debate is **still open** — and now suddenly practical.

Why the field split: replies

- **Systems reply**: the whole *system* understands, not just you.
Searle: memorize the full rulebook — do you understand now?
- **Robot reply**: ground the system in a body and sensors.
Searle: still symbol manipulation, just with richer inputs.
- **Brain simulator**: simulate actual neurons.
Searle: simulating water doesn't make you wet.
- **Functionalists**: behavior *is* mind. None of these replies concede the point.

Language Modeling

Suppose a system could finish any sentence
the way a **knowledgeable person** would.

Suppose a system could finish any sentence
the way a **knowledgeable person** would.

What could you do with that?

What Text Comes Next?

Prompt: “The dim lighting, lukewarm food, and indifferent service made for a . . .”

What Text Comes Next?

Prompt: “The dim lighting, lukewarm food, and indifferent service made for a...”

Completion A

“... a **delightful** evening. The chef clearly cares about every dish.”

What Text Comes Next?

Prompt: “The dim lighting, lukewarm food, and indifferent service made for a...”

Completion A

“... a **delightful** evening. The chef clearly cares about every dish.”

Completion B

“... The mitochondria is the powerhouse of the cell.”

Now a Harder Question

Prompt: “The dim lighting, lukewarm food, and indifferent service made for a...”

Completion A

“... a **delightful** evening. The chef clearly cares about every dish.”

Now a Harder Question

Prompt: “The dim lighting, lukewarm food, and indifferent service made for a...”

Completion A

“... a **delightful** evening. The chef clearly cares about every dish.”

Completion B

“... a **disappointing** evening. I would not return.”

The Key Insight

The prompt described dim lighting, lukewarm food, and indifferent service.

The prompt described dim lighting, lukewarm food, and indifferent service.

A model that truly understands language
must assign “disappointing” far higher probability than “delightful”
given that context.

The Key Insight

The prompt described dim lighting, lukewarm food, and indifferent service.

A model that truly understands language
must assign “disappointing” far higher probability than “delightful”
given that context.

If it can do *this*, it can classify sentiment, answer questions,
reason clinically — **almost any task is a completion task.**

Language Modeling as a Probability Problem

By the **chain rule of probability**, any sequence probability factors into next-word predictions:

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

Language Modeling as a Probability Problem

The key reduction

To model any sequence, **all we need** is to predict the next word given everything before it.

- (1) Score a sequence
- (2) Predict the next word
- (3) Generate new text

Language Modeling as a Probability Problem

The key reduction

To model any sequence, **all we need** is to predict the next word given everything before it.

- (1) Score a sequence (2) Predict the next word (3) Generate new text

From our review prompt, a trained model should assign:

$$P(\text{"disappointing"} \mid \text{context}) \gg P(\text{"delightful"} \mid \text{context})$$

The model must learn — from data — to assign high probability to contextually appropriate words.

From Naïve Lookup to Transformers

A Naïve Approach: Just Look It Up

Simplest possible idea: given a context, search a large corpus for *exact matches* and see what word came next. No model — just a lookup table.

A Naïve Approach: Just Look It Up

Simplest possible idea: given a context, search a large corpus for *exact matches* and see what word came next. No model — just a lookup table.

Shorter queries work fine

“the” — billions of matches.

“the film” — millions of matches.

“thoroughly disappointing” — many matches.

A Naïve Approach: Just Look It Up

Simplest possible idea: given a context, search a large corpus for *exact matches* and see what word came next. No model — just a lookup table.

Shorter queries work fine

“the” — billions of matches.

“the film” — millions of matches.

“thoroughly disappointing” — many matches.

Longer queries: zero matches

*“The dim lighting, lukewarm food, and
indifferent service made for a thoroughly”*

Even with 10^{12} tokens of text (CommonCrawl scale), this context appears **exactly zero times** in any corpus.

A Naïve Approach: Just Look It Up

Simplest possible idea: given a context, search a large corpus for *exact matches* and see what word came next. No model — just a lookup table.

Shorter queries work fine

“the” — billions of matches.

“the film” — millions of matches.

“thoroughly disappointing” — many matches.

Longer queries: zero matches

“The dim lighting, lukewarm food, and indifferent service made for a thoroughly”

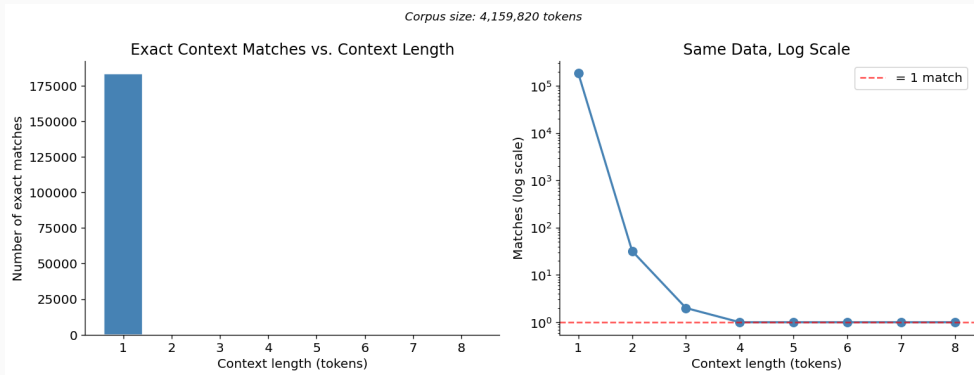
Even with 10^{12} tokens of text (CommonCrawl scale), this context appears **exactly zero times** in any corpus.

Why does this happen?

The space of possible contexts is **combinatorially enormous**. Even a modest sentence is effectively unique. We cannot rely on having seen it before.

❖ **Notebook §1** — Empirically verify this on a real corpus. Measure exact-match counts as context length increases.

§1 Notebook: Match Counts Drop to Zero Fast



PubMed corpus (4.2M tokens). Even 2-word queries are vanishingly rare; 4-word queries functionally never repeat.

N-gram Models: Use Only the Last n Words

Idea: if the full context never matches, use only the *last* $n-1$ words (Markov assumption):

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

Estimate from counts in a corpus. Simple, fast, and surprisingly effective for small n .

N-gram Models: Use Only the Last n Words

Idea: if the full context never matches, use only the *last* $n-1$ words (Markov assumption):

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

Estimate from counts in a corpus. Simple, fast, and surprisingly effective for small n .

Unigram ($n = 1$): $P(w_t)$

*“the patient of sessions sirna developed
injection in gray prescription phosphenes
ivu. . .”*

N-gram Models: Use Only the Last n Words

Idea: if the full context never matches, use only the *last* $n-1$ words (Markov assumption):

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

Estimate from counts in a corpus. Simple, fast, and surprisingly effective for small n .

Unigram ($n = 1$): $P(w_t)$

*“the patient of sessions sirna developed
injection in gray prescription phosphenes
ivu. . .”*

Bigram ($n = 2$): $P(w_t \mid w_{t-1})$

*“the patient resulting in the results from
additional dose pantoprazole on production
of patients with respect to. . .”*

N-gram Models: Use Only the Last n Words

Idea: if the full context never matches, use only the *last* $n-1$ words (Markov assumption):

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

Estimate from counts in a corpus. Simple, fast, and surprisingly effective for small n .

Unigram ($n = 1$): $P(w_t)$

“the patient of sessions sirna developed injection in gray prescription phosphenes ivu. . .”

Bigram ($n = 2$): $P(w_t \mid w_{t-1})$

“the patient resulting in the results from additional dose pantoprazole on production of patients with respect to. . .”

Trigram ($n = 3$): $P(w_t \mid w_{t-2}, w_{t-1})$

“the patient underwent an integrated and linked to the existing drg as indirect support for the determination of malignancy. . .”

N-gram Models: Use Only the Last n Words

Idea: if the full context never matches, use only the *last* $n-1$ words (Markov assumption):

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

Estimate from counts in a corpus. Simple, fast, and surprisingly effective for small n .

Unigram ($n = 1$): $P(w_t)$

“the patient of sessions sirna developed injection in gray prescription phosphenes ivu. . .”

Bigram ($n = 2$): $P(w_t \mid w_{t-1})$

“the patient resulting in the results from additional dose pantoprazole on production of patients with respect to. . .”

Trigram ($n = 3$): $P(w_t \mid w_{t-2}, w_{t-1})$

“the patient underwent an integrated and linked to the existing drg as indirect support for the determination of malignancy. . .”

5-gram ($n = 5$)

“the patient was adherent to treatment he had decreased level of consciousness she underwent whole body positron emission tomography. . .”

Discussion question:

“The patient’s prior visit showed elevated creatinine and signs of early renal insufficiency. At this visit, we observed normal creatinine but new peripheral edema and rising BNP. Taking both their recent and past visits into account, we can conclude the patient’s condition has _____.”

Discussion question:

“The patient’s prior visit showed elevated creatinine and signs of early renal insufficiency. At this visit, we observed normal creatinine but new peripheral edema and rising BNP. Taking both their recent and past visits into account, we can conclude the patient’s condition has _____.”

What fills the blank? What *evidence* does it depend on?

The answer requires the full context

*“... the patient's condition has **evolved** / **worsened** / **shifted**.”*

The right completion depends on *both* visits: improving creatinine from the prior visit *and* new edema/BNP from this visit. Neither alone is sufficient.

Long-Range Dependencies

The answer requires the full context

“... the patient's condition has **evolved** / **worsened** / **shifted**.”

The right completion depends on *both* visits: improving creatinine from the prior visit *and* new edema/BNP from this visit. Neither alone is sufficient.

What n-gram models predict at the blank

2-gram top-5: the, in, by, for, using

3-gram: no matching context found

5-gram: no matching context found

The key word “patient” is more than 5 tokens back — simply **out of window** at any practical n .

Long-Range Dependencies

The answer requires the full context

“... the patient's condition has **evolved** / **worsened** / **shifted**.”

The right completion depends on *both* visits: improving creatinine from the prior visit *and* new edema/BNP from this visit. Neither alone is sufficient.

What n-gram models predict at the blank

2-gram top-5: the, in, by, for, using

3-gram: no matching context found

5-gram: no matching context found

The key word “patient” is more than 5 tokens back — simply **out of window** at any practical n .

The catch

Increasing n would help — but the parameter space grows so fast that large n is physically unreachable. We're stuck.

The Combinatorial Abyss: Why Context Scales Are Unreachable

Context length	Possible contexts (V^n)	Comparison
1 token	5×10^4	Trivial

The Combinatorial Abyss: Why Context Scales Are Unreachable

Context length	Possible contexts (V^n)	Comparison
1 token	5×10^4	Trivial
2 tokens	2.5×10^9	> world population

The Combinatorial Abyss: Why Context Scales Are Unreachable

Context length	Possible contexts (V^n)	Comparison
1 token	5×10^4	Trivial
2 tokens	2.5×10^9	> world population
4 tokens	6.3×10^{18}	$\approx$ grains of sand on Earth's beaches

The Combinatorial Abyss: Why Context Scales Are Unreachable

Context length	Possible contexts (V^n)	Comparison
1 token	5×10^4	Trivial
2 tokens	2.5×10^9	> world population
4 tokens	6.3×10^{18}	$\approx$ grains of sand on Earth's beaches
6 tokens	1.6×10^{28}	$\gg$ stars in the observable universe

The Combinatorial Abyss: Why Context Scales Are Unreachable

Context length	Possible contexts (V^n)	Comparison
1 token	5×10^4	Trivial
2 tokens	2.5×10^9	> world population
4 tokens	6.3×10^{18}	$\approx$ grains of sand on Earth's beaches
6 tokens	1.6×10^{28}	$\gg$ stars in the observable universe
10 tokens	$\approx 10^{47}$	$\gg$ atoms in the Milky Way

The Combinatorial Abyss: Why Context Scales Are Unreachable

Context length	Possible contexts (V^n)	Comparison
1 token	5×10^4	Trivial
2 tokens	2.5×10^9	> world population
4 tokens	6.3×10^{18}	$\approx$ grains of sand on Earth's beaches
6 tokens	1.6×10^{28}	$\gg$ stars in the observable universe
10 tokens	$\approx 10^{47}$	$\gg$ atoms in the Milky Way
20 tokens	$\approx 10^{94}$	$\gg$ atoms in the observable universe

The Combinatorial Abyss: Why Context Scales Are Unreachable

Context length	Possible contexts (V^n)	Comparison
1 token	5×10^4	Trivial
2 tokens	2.5×10^9	> world population
4 tokens	6.3×10^{18}	$\approx$ grains of sand on Earth's beaches
6 tokens	1.6×10^{28}	$\gg$ stars in the observable universe
10 tokens	$\approx 10^{47}$	$\gg$ atoms in the Milky Way
20 tokens	$\approx 10^{94}$	$\gg$ atoms in the observable universe
128K tokens	$\approx 10^{600,000}$	Incomprehensibly beyond physical reality

The Combinatorial Abyss: Why Context Scales Are Unreachable

Context length	Possible contexts (V^n)	Comparison
1 token	5×10^4	Trivial
2 tokens	2.5×10^9	> world population
4 tokens	6.3×10^{18}	$\approx$ grains of sand on Earth's beaches
6 tokens	1.6×10^{28}	$\gg$ stars in the observable universe
10 tokens	$\approx 10^{47}$	$\gg$ atoms in the Milky Way
20 tokens	$\approx 10^{94}$	$\gg$ atoms in the observable universe
128K tokens	$\approx 10^{600,000}$	Incomprehensibly beyond physical reality

The conclusion

We cannot enumerate all contexts. We *cannot* use n-grams at the scales modern LLMs operate at. We need a model that **generalizes without memorizing** — and one that can handle arbitrarily long context.

The Transformer

What We Need From a Language Model

We need a learned function from context to next-word probabilities. What properties must it have?

What We Need From a Language Model

We need a learned function from context to next-word probabilities. What properties must it have?

1. **Variable-length input** — sentences are not all the same length.

What We Need From a Language Model

We need a learned function from context to next-word probabilities. What properties must it have?

1. **Variable-length input** — sentences are not all the same length.
2. **Long-range dependencies** — context from far away must be reachable.

What We Need From a Language Model

We need a learned function from context to next-word probabilities. What properties must it have?

1. **Variable-length input** — sentences are not all the same length.
2. **Long-range dependencies** — context from far away must be reachable.
3. **Valid probability output** — a proper distribution over the vocabulary at each step.

What We Need From a Language Model

We need a learned function from context to next-word probabilities. What properties must it have?

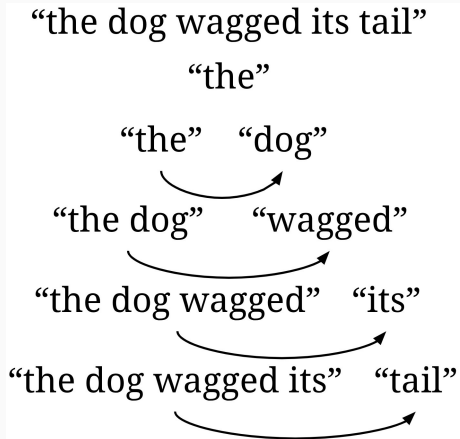
1. **Variable-length input** — sentences are not all the same length.
2. **Long-range dependencies** — context from far away must be reachable.
3. **Valid probability output** — a proper distribution over the vocabulary at each step.
4. **Order sensitivity** — *“dog bites man”* $\neq$ *“man bites dog”*.

What We Need From a Language Model

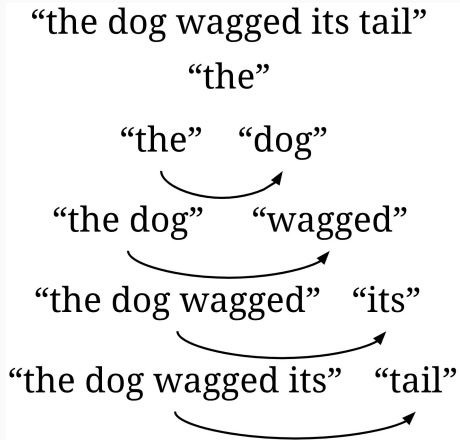
We need a learned function from context to next-word probabilities. What properties must it have?

1. **Variable-length input** — sentences are not all the same length.
2. **Long-range dependencies** — context from far away must be reachable.
3. **Valid probability output** — a proper distribution over the vocabulary at each step.
4. **Order sensitivity** — *“dog bites man”* $\neq$ *“man bites dog”*.
5. **Scalable training** — must work with trillions of tokens and billions of parameters.

A Language Model: The Big Picture



A Language Model: The Big Picture



Each prefix predicts the next word. We append it and repeat — generation is just next-token prediction, applied autoregressively.

Step 1: Words $\rightarrow$ Vectors (Embeddings)

The problem

Neural networks operate on real-valued vectors. Words are discrete symbols. We need a bridge.

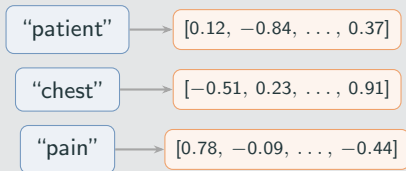
Step 1: Words $\rightarrow$ Vectors (Embeddings)

The problem

Neural networks operate on real-valued vectors. Words are discrete symbols. We need a bridge.

The embedding lookup

Every token in the vocabulary gets a trainable vector $\mathbf{e} \in \mathbb{R}^d$ (typically $d = 512\text{--}4096$).



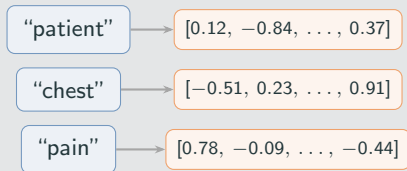
Step 1: Words $\rightarrow$ Vectors (Embeddings)

The problem

Neural networks operate on real-valued vectors. Words are discrete symbols. We need a bridge.

The embedding lookup

Every token in the vocabulary gets a trainable vector $\mathbf{e} \in \mathbb{R}^d$ (typically $d = 512\text{--}4096$).

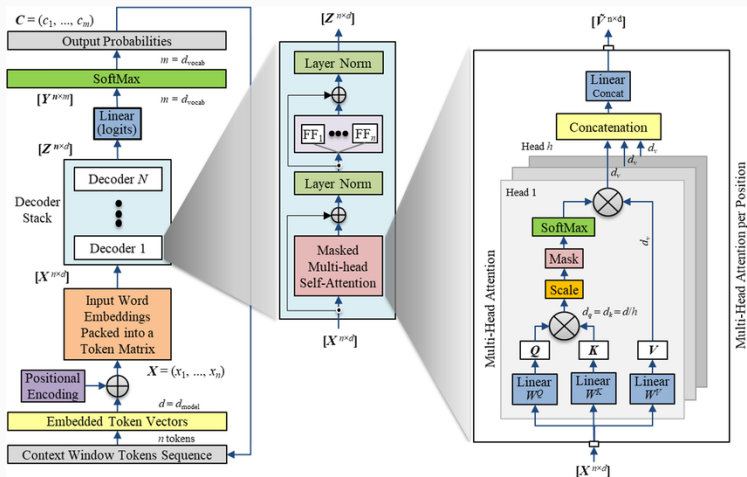


What embeddings learn

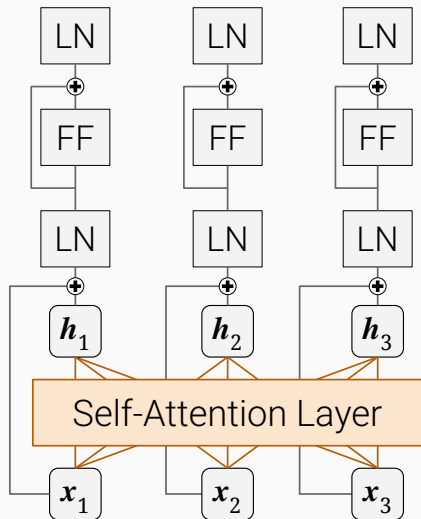
Directions in embedding space encode meaning:

- Similar words cluster together
- Analogies become vector arithmetic
- Medical synonyms get similar vectors after training on clinical text

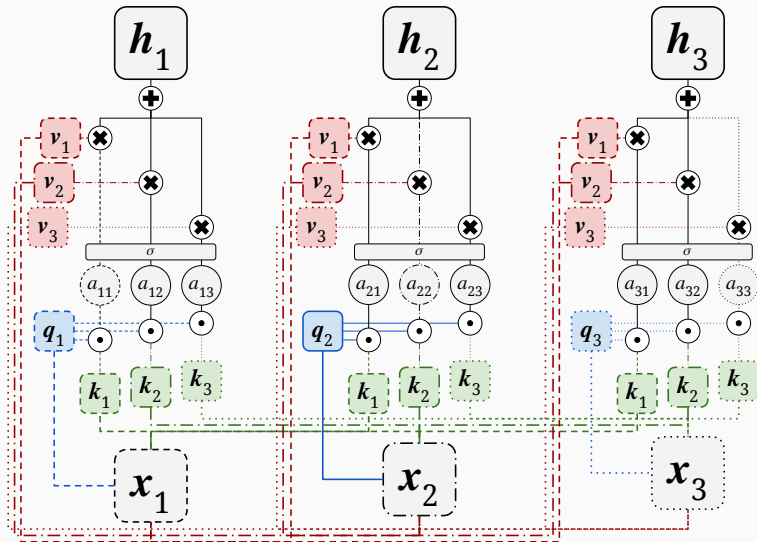
The Decoder-Only Transformer (GPT Family)



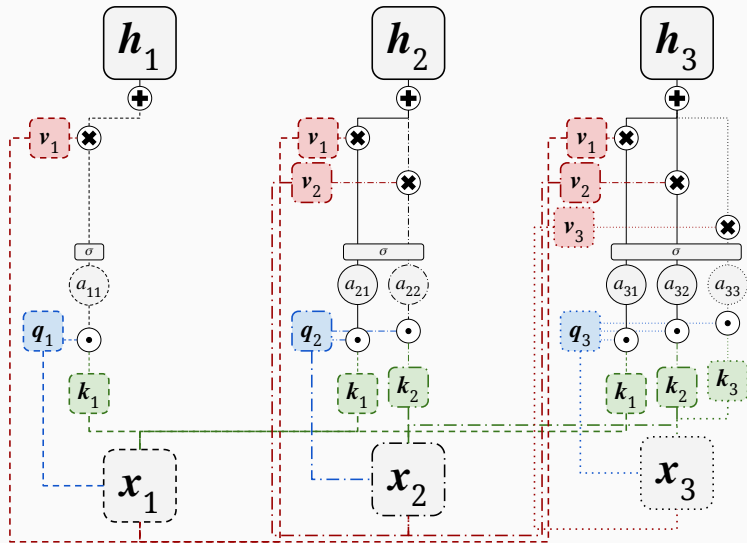
Each Transformer Layer



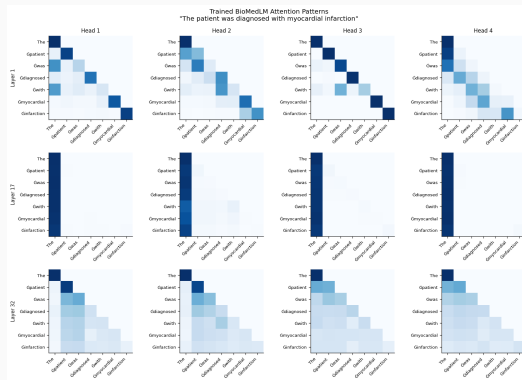
The Attention Mechanism



The Attention Mechanism



§5 Notebook: BioMedLM Attention on a Clinical Sentence



12 heads (3 layers) on "The patient was diagnosed with myocardial infarction." Early: local/positional.
Later: semantic — does any head link "infarction" back to "myocardial"?

Go Deeper: Interactive Resources

Transformer Explainer

<https://poloclub.github.io/transformer-explainer/>

Live in-browser GPT-2 visualization — step through tokenization, embeddings, every attention head, and next-token prediction in real time. Great to open in class.

The Illustrated Transformer

Jay Alammar's canonical visual walkthrough. Best introduction to attention; used at Stanford, MIT, CMU, Princeton.

<https://jalammar.github.io/illustrated-transformer/>

Suggested: read Alammar, then open the Explainer with a prompt of your choosing, then revisit the math in the slides and Notebook §3.

Three Intuitions for Attention

1. Probabilistic expectation

$$\text{out}_t = \mathbb{E}_{s \sim \alpha_t} [\mathbf{v}_s]$$

α_{ts} is a **distribution** over positions. Output is the expected value vector under that distribution.

Three Intuitions for Attention

1. Probabilistic expectation

$$\text{out}_t = \mathbb{E}_{s \sim \alpha_t} [\mathbf{v}_s]$$

α_{ts} is a **distribution** over positions. Output is the expected value vector under that distribution.

2. Soft retrieval

Queries ask a question.

Key are index entries.

Values hold the content.

Like a database lookup —
but **differentiable**:
gradients flow through the
retrieval.

Three Intuitions for Attention

1. Probabilistic expectation

$$\text{out}_t = \mathbb{E}_{s \sim \alpha_t} [\mathbf{v}_s]$$

α_{ts} is a **distribution** over positions. Output is the expected value vector under that distribution.

2. Soft retrieval

Queries ask a question.

Key are index entries.

Values hold the content.

Like a database lookup —
but **differentiable**:
gradients flow through the
retrieval.

3. Token self-revision

Each token has a current representation. It *exposes* that to the rest of the sequence, then *looks around* at everyone else to decide: given what I see, how should I revise my own representation for the next layer?

Three Intuitions for Attention

1. Probabilistic expectation

$$\text{out}_t = \mathbb{E}_{s \sim \alpha_t} [\mathbf{v}_s]$$

α_{ts} is a **distribution** over positions. Output is the expected value vector under that distribution.

2. Soft retrieval

Queries ask a question.

Key are index entries.

Values hold the content.

Like a database lookup — but **differentiable**: gradients flow through the retrieval.

3. Token self-revision

Each token has a current representation. It *exposes* that to the rest of the sequence, then *looks around* at everyone else to decide: given what I see, how should I revise my own representation for the next layer?

$$\alpha_{ts} = \text{softmax}\left(\frac{\mathbf{q}_t^\top \mathbf{k}_s}{\sqrt{d_k}}\right)$$

(scaling by $\sqrt{d_k}$ prevents softmax saturation)

Inside a Transformer Block

Each block applies two sub-layers with **residual connections**:

Inside a Transformer Block

Each block applies two sub-layers with **residual connections**:

$$\mathbf{x}' = \mathbf{x} + \text{MultiHeadAttention}(\text{LayerNorm}(\mathbf{x}))$$

Inside a Transformer Block

Each block applies two sub-layers with **residual connections**:

$$\mathbf{x}' = \mathbf{x} + \text{MultiHeadAttention}(\text{LayerNorm}(\mathbf{x}))$$

$$\mathbf{x}'' = \mathbf{x}' + \text{FeedForward}(\text{LayerNorm}(\mathbf{x}'))$$

Inside a Transformer Block

Each block applies two sub-layers with **residual connections**:

$$\mathbf{x}' = \mathbf{x} + \text{MultiHeadAttention}(\text{LayerNorm}(\mathbf{x}))$$

$$\mathbf{x}'' = \mathbf{x}' + \text{FeedForward}(\text{LayerNorm}(\mathbf{x}'))$$

Why residuals?

Each block **adds** a correction to a running representation — it never replaces it. This lets gradients flow cleanly through 100+ layers and prevents vanishing gradient problems.

Inside a Transformer Block

Each block applies two sub-layers with **residual connections**:

$$\mathbf{x}' = \mathbf{x} + \text{MultiHeadAttention}(\text{LayerNorm}(\mathbf{x}))$$

$$\mathbf{x}'' = \mathbf{x}' + \text{FeedForward}(\text{LayerNorm}(\mathbf{x}'))$$

Why residuals?

Each block **adds** a correction to a running representation — it never replaces it. This lets gradients flow cleanly through 100+ layers and prevents vanishing gradient problems.

Why layer norm?

Normalizes activations before each sub-layer (Pre-LN variant). Stabilizes magnitudes during training and reduces sensitivity to learning rate choice.

Positional Encoding: Teaching Order Without Recurrence

The problem

Attention is **permutation-invariant**: without extra information, “*dog bites man*” and “*man bites dog*” are indistinguishable.

Positional Encoding: Teaching Order Without Recurrence

The problem

Attention is **permutation-invariant**: without extra information, “*dog bites man*” and “*man bites dog*” are indistinguishable.

Sinusoidal encoding (Vaswani et al., 2017)

$$\text{PE}_{t,2i} = \sin\left(\frac{t}{10000^{2i/d}}\right)$$

A fixed vector added to each token embedding. Different frequencies encode position at different scales — nearby tokens share low-frequency components; exact positions differ at high frequencies.

Modern Positional Encoding Schemes

Learned absolute PE

(GPT-2, BERT)

One trainable embedding per position. Simple, effective within training length, but does not extrapolate beyond it.

Modern Positional Encoding Schemes

Learned absolute PE

(GPT-2, BERT)

One trainable embedding per position. Simple, effective within training length, but does not extrapolate beyond it.

RoPE — Rotary Position Embedding

(LLaMA, GPT-NeoX)

Encodes *relative* position by rotating Q and K vectors before the dot product. Better length generalization than absolute PE; now the dominant approach.

Modern Positional Encoding Schemes

Learned absolute PE

(GPT-2, BERT)

One trainable embedding per position. Simple, effective within training length, but does not extrapolate beyond it.

RoPE — Rotary Position Embedding

(LLaMA, GPT-NeoX)

Encodes *relative* position by rotating Q and K vectors before the dot product. Better length generalization than absolute PE; now the dominant approach.

Key limitation

Position is an **indirect** inductive bias regardless of scheme — the model must *learn* to use it. Unlike RNNs where order is architecturally enforced, this creates measurable degradation on long contexts (revisited in Open Challenges).

What Made Transformers Actually Work

Architectural

- Scaled dot-product attention ($\div \sqrt{d_k}$)
- Multi-head attention (parallel specialization)
- Residual connections (deep training)
- Layer normalization (stability)
- Large FFN sub-layer ($4\times$ hidden dim)
- Byte-pair encoding (open vocabulary)

What Made Transformers Actually Work

Architectural

- Scaled dot-product attention ($\div \sqrt{d_k}$)
- Multi-head attention (parallel specialization)
- Residual connections (deep training)
- Layer normalization (stability)
- Large FFN sub-layer ($4\times$ hidden dim)
- Byte-pair encoding (open vocabulary)

Training and alignment

- **Scale**: trillions of tokens, billions of params
- **Instruction fine-tuning**: task and format following
- **RLHF**: human preferences $\rightarrow$ reward model $\rightarrow$ PPO
- **DPO**: direct preference optimization
- **Constitutional AI**: self-critique loop

What Made Transformers Actually Work

Architectural

- Scaled dot-product attention ($\div \sqrt{d_k}$)
- Multi-head attention (parallel specialization)
- Residual connections (deep training)
- Layer normalization (stability)
- Large FFN sub-layer ($4\times$ hidden dim)
- Byte-pair encoding (open vocabulary)

Training and alignment

- **Scale**: trillions of tokens, billions of params
- **Instruction fine-tuning**: task and format following
- **RLHF**: human preferences $\rightarrow$ reward model $\rightarrow$ PPO
- **DPO**: direct preference optimization
- **Constitutional AI**: self-critique loop

No single ingredient is magic. The combination — at scale — produced emergent capability.

Mathematical Challenge: Quadratic Attention Cost

Computing attention requires **all** $T \times T$ pairwise scores:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

Complexity: $\mathcal{O}(T^2d)$ time and $\mathcal{O}(T^2)$ memory.

Mathematical Challenge: Quadratic Attention Cost

Computing attention requires **all** $T \times T$ pairwise scores:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

Complexity: $\mathcal{O}(T^2d)$ time and $\mathcal{O}(T^2)$ memory.

Practical impact

- $T = 1,024$: $\sim 1\text{M}$ attention pairs
- $T = 8,192$: $\sim 67\text{M}$ pairs
- $T = 128,000$: $\sim 16\text{B}$ pairs

Context window = hard engineering constraint.

Mathematical Challenge: Quadratic Attention Cost

Computing attention requires **all** $T \times T$ pairwise scores:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

Complexity: $\mathcal{O}(T^2d)$ time and $\mathcal{O}(T^2)$ memory.

Practical impact

- $T = 1,024$: $\sim 1\text{M}$ attention pairs
- $T = 8,192$: $\sim 67\text{M}$ pairs
- $T = 128,000$: $\sim 16\text{B}$ pairs

Context window = hard engineering constraint.

Active solutions

- **FlashAttention:** I/O-aware; same $\mathcal{O}(T^2)$ ops, 2–4 $\times$ faster in practice.
- **Sliding window:** Longformer, BigBird — local $\mathcal{O}(T)$ attention.
- **State space models:** Mamba, S4 — $\mathcal{O}(T)$ with recurrent structure.

Challenge: Exact Decoding is Intractable

Greedy decoding: at each step pick $\arg \max_v P(w_{t+1} = v \mid \text{context})$.

But the **globally most-likely sequence** is not the product of greedy local choices.

Challenge: Exact Decoding is Intractable

Greedy decoding: at each step pick $\arg \max_v P(w_{t+1} = v \mid \text{context})$.

But the **globally most-likely sequence** is not the product of greedy local choices.

Decoding strategies

- **Beam search:** keep top- k sequences; better quality
- **Temperature sampling:** rescale logits; diverse, creative
- **Nucleus (top- p):** sample from smallest set covering probability p

Challenge: Exact Decoding is Intractable

Greedy decoding: at each step pick $\arg \max_v P(w_{t+1} = v \mid \text{context})$.

But the **globally most-likely sequence** is not the product of greedy local choices.

Decoding strategies

- **Beam search:** keep top- k sequences; better quality
- **Temperature sampling:** rescale logits; diverse, creative
- **Nucleus (top- p):** sample from smallest set covering probability p

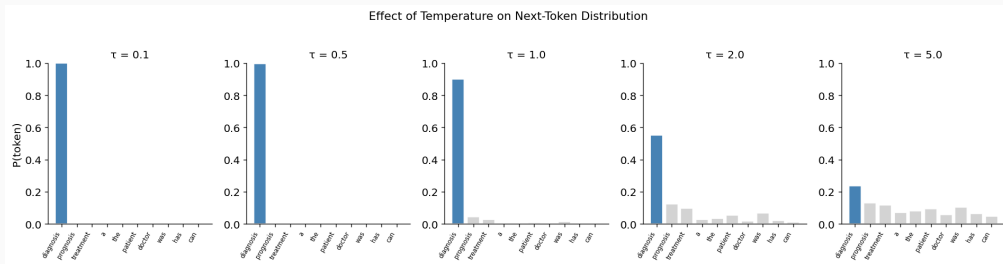
Do we want the “most likely”?

“Does this patient have rare cancer X?” — the maximally likely answer is almost always *No*, even when the patient is far more likely than baseline to have it.

Maximum likelihood $\neq$ calibrated inference.

❖ **Notebook §4** — Compare greedy, temperature, and nucleus sampling on a clinical prompt. Observe output variance.

§4 Notebook: How Temperature Reshapes the Distribution



Low temperature ($\tau \rightarrow 0$) collapses mass onto the mode — greedy decoding. High temperature flattens the distribution, increasing diversity at the cost of coherence.

§5 Notebook: Five Strategies, Four Diagnoses

Prompt: *“The patient presents with acute chest pain and shortness of breath. The most likely diagnosis is”*

§5 Notebook: Five Strategies, Four Diagnoses

Prompt: *“The patient presents with acute chest pain and shortness of breath. The most likely diagnosis is”*

Greedy

“... **pulmonary embolism.**” (then generates MCQ-style artifacts from PubMed training data)

§5 Notebook: Five Strategies, Four Diagnoses

Prompt: *“The patient presents with acute chest pain and shortness of breath. The most likely diagnosis is”*

Greedy

“... **pulmonary embolism.**” (then generates MCQ-style artifacts from PubMed training data)

Beam search ($k = 4$)

“... **acute coronary syndrome (ACS).**”

§5 Notebook: Five Strategies, Four Diagnoses

Prompt: *“The patient presents with acute chest pain and shortness of breath. The most likely diagnosis is”*

Greedy

“... **pulmonary embolism.**” (then generates MCQ-style artifacts from PubMed training data)

Beam search ($k = 4$)

“... **acute coronary syndrome (ACS).**”

Temperature $\tau = 0.7$

“... **pulmonary embolism.** Her history includes hypertension, hypercholesterolemia, and remote DVT...” (most fluent)

§5 Notebook: Five Strategies, Four Diagnoses

Prompt: *“The patient presents with acute chest pain and shortness of breath. The most likely diagnosis is”*

Greedy

“... **pulmonary embolism.**” (then generates MCQ-style artifacts from PubMed training data)

Beam search ($k = 4$)

“... **acute coronary syndrome (ACS).**”

Temperature $\tau = 0.7$

“... **pulmonary embolism.** Her history includes hypertension, hypercholesterolemia, and remote DVT...” (most fluent)

Nucleus $p = 0.9$

“... **pericarditis.**”

Top- k ($k = 50$)

“... **coronary artery disease with acute ischemia.**”

§5 Notebook: Five Strategies, Four Diagnoses

Prompt: *"The patient presents with acute chest pain and shortness of breath. The most likely diagnosis is"*

Greedy

"... **pulmonary embolism.**" (then generates MCQ-style artifacts from PubMed training data)

Beam search ($k = 4$)

"... **acute coronary syndrome (ACS).**"

Temperature $\tau = 0.7$

"... **pulmonary embolism.** Her history includes hypertension, hypercholesterolemia, and remote DVT..." (**most fluent**)

Nucleus $p = 0.9$

"... **pericarditis.**"

Top- k ($k = 50$)

"... **coronary artery disease with acute ischemia.**"

Same model, same prompt, five different answers. In clinical deployment: which do you report? How do you know which is right?

The LLM Ecosystem

Proprietary APIs

- GPT (OpenAI)
- Claude (Anthropic)
- Gemini (Google)

Easy to use; data sent to third-party servers.

The LLM Landscape

Proprietary APIs

- GPT (OpenAI)
- Claude (Anthropic)
- Gemini (Google)

Easy to use; data sent to third-party servers.

Open weights

- LLaMA (Meta)
- Mistral / Mixtral
- Phi-3 (Microsoft)
- Qwen, Gemma, ...

Downloadable.

Customizable. Local deployment possible.

The LLM Landscape

Proprietary APIs

- GPT (OpenAI)
- Claude (Anthropic)
- Gemini (Google)

Easy to use; **data sent to third-party servers.**

Open weights

- LLaMA (Meta)
- Mistral / Mixtral
- Phi-3 (Microsoft)
- Qwen, Gemma, ...

Downloadable.

Customizable. Local deployment possible.

Health AI constraint

For clinical data (HIPAA), **local deployment** of open-weight models may be required. Proprietary APIs send PHI to external servers.

Infrastructure

HuggingFace Hub, Ollama, vLLM, SGLang

HuggingFace: The GitHub of AI Models

What it provides

- **Model Hub:** 500K+ pre-trained models
- **Datasets:** 100K+ curated datasets
- **Transformers:** one unified API for all architectures
- **PEFT:** LoRA, prefix tuning, adapters
- **Accelerate:** multi-GPU / distributed training

HuggingFace: The GitHub of AI Models

What it provides

- **Model Hub:** 500K+ pre-trained models
- **Datasets:** 100K+ curated datasets
- **Transformers:** one unified API for all architectures
- **PEFT:** LoRA, prefix tuning, adapters
- **Accelerate:** multi-GPU / distributed training

Two lines to a language model

```
from transformers import pipeline  
gen = pipeline(  
    "text-generation", model="gpt2")  
gen("The patient presents",  
    max_new_tokens=40)
```

❖ **Notebook §5** — Load GPT-2 via HuggingFace. Inspect medical tokenization. Generate text. Visualize trained attention. Compute perplexity.

The memory problem

LLaMA 3 70B at full precision: **140 GB**.

An A100 GPU: 80 GB. RTX 4090: 24 GB.

Quantization compresses weights:

- fp16 / bf16: $\sim 2\times$ compression, tiny quality loss
- INT8: $\sim 4\times$ compression (bitsandbytes)
- GGUF 4-bit (Ollama): 70B fits on a laptop

Running LLMs Locally: Quantization and LoRA

The memory problem

LLaMA 3 70B at full precision: **140 GB**.

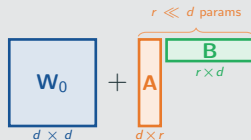
An A100 GPU: 80 GB. RTX 4090: 24 GB.

Quantization compresses weights:

- fp16 / bf16: $\sim 2\times$ compression, tiny quality loss
- INT8: $\sim 4\times$ compression (bitsandbytes)
- GGUF 4-bit (Ollama): 70B fits on a laptop

LoRA: fine-tune without full weights

Freeze $\mathbf{W}_0 \in \mathbb{R}^{d \times d}$. Add a low-rank correction $\mathbf{AB}$:



$r=8, d=4096$: $\mathbf{AB}$ has $2 \times 8 \times 4096 \approx 65\text{K}$ params vs. $4096^2 \approx 17\text{M}$ — $>99\%$ savings.

Open Challenges

Challenge: Hallucination

What is hallucination?

LLMs generate confident, fluent, plausible-sounding text that is **factually wrong**.

Why? The model optimizes for likely text, not true text:

$$\arg \max P(\mathbf{w}_{1:T}) \neq \arg \max P(\mathbf{w}_{1:T} \mid \mathbf{w} \text{ is true})$$

There is no explicit truth signal in next-token prediction training.

Challenge: Hallucination

What is hallucination?

LLMs generate confident, fluent, plausible-sounding text that is **factually wrong**.

Why? The model optimizes for likely text, not true text:

$$\arg \max P(\mathbf{w}_{1:T}) \neq \arg \max P(\mathbf{w}_{1:T} \mid \mathbf{w} \text{ is true})$$

There is no explicit truth signal in next-token prediction training.

Health AI consequences

- Fabricated drug interactions
- Incorrect dosages, stated confidently
- Non-existent guidelines cited
- False patient summaries

Challenge: Hallucination

What is hallucination?

LLMs generate confident, fluent, plausible-sounding text that is **factually wrong**.

Why? The model optimizes for likely text, not true text:

$$\arg \max P(\mathbf{w}_{1:T}) \neq \arg \max P(\mathbf{w}_{1:T} \mid \mathbf{w} \text{ is true})$$

There is no explicit truth signal in next-token prediction training.

Health AI consequences

- Fabricated drug interactions
- Incorrect dosages, stated confidently
- Non-existent guidelines cited
- False patient summaries

Mitigations

- **RAG**: ground outputs in retrieved facts
- Forced source attribution
- Train on “I don’t know”
- Human-in-the-loop verification

Chain of Thought (Wei et al., 2022)

Prompting models to emit **intermediate steps** dramatically improves multi-step reasoning:

$$Q \rightarrow \text{step}_1 \rightarrow \cdots \rightarrow \text{step}_k \rightarrow A$$

Extensions: Tree of Thought, Graph of Thought, self-consistency.

Chain of Thought (Wei et al., 2022)

Prompting models to emit **intermediate steps** dramatically improves multi-step reasoning:

$$Q \rightarrow \text{step}_1 \rightarrow \cdots \rightarrow \text{step}_k \rightarrow A$$

Extensions: Tree of Thought, Graph of Thought, self-consistency.

Test-time compute

OpenAI o1, DeepSeek-R1: scale reasoning at **inference** rather than only at training time. Models “think longer” on hard problems.

Chain of Thought (Wei et al., 2022)

Prompting models to emit **intermediate steps** dramatically improves multi-step reasoning:

$$Q \rightarrow \text{step}_1 \rightarrow \dots \rightarrow \text{step}_k \rightarrow A$$

Extensions: Tree of Thought, Graph of Thought, self-consistency.

Test-time compute

OpenAI o1, DeepSeek-R1: scale reasoning at **inference** rather than only at training time. Models “think longer” on hard problems.

Architectural frontier

- **MoE** (Mixture of Experts): sparse activation, only a fraction of parameters fire per token (Mixtral, GPT-4, DeepSeek-V3)
- **SSMs** (Mamba, RWKV): $\mathcal{O}(T)$ alternatives to attention
- **RAG**: retrieve relevant docs at inference time
- **Agentic LLMs**: tool use, code execution, multi-agent pipelines

We Built the Chinese Room

What we built

We Built the Chinese Room

What we built

- A conditional probability estimator over token sequences

We Built the Chinese Room

What we built

- A conditional probability estimator over token sequences
- Trained on the combinatorially vast space of human-written text

We Built the Chinese Room

What we built

- A conditional probability estimator over token sequences
- Trained on the combinatorially vast space of human-written text
- Using attention to dynamically route context across any distance

We Built the Chinese Room

What we built

- A conditional probability estimator over token sequences
- Trained on the combinatorially vast space of human-written text
- Using attention to dynamically route context across any distance
- With *no* explicit world model, no causal structure, no truth signal

ChatGPT, Claude, Gemini: billions of parameters, no grounding.

Searle's thought experiment **became a product.**

We Built the Chinese Room

What we built

- A conditional probability estimator over token sequences
- Trained on the combinatorially vast space of human-written text
- Using attention to dynamically route context across any distance
- With *no* explicit world model, no causal structure, no truth signal

ChatGPT, Claude, Gemini: billions of parameters, no grounding.

Searle's thought experiment **became a product.**

And it works.

It passes the bar exam.

It diagnoses rare diseases from notes.

It writes code, poetry, legal briefs.

Does it understand any of it?

Searle said: impossible.

We built it anyway.

The question is now empirical — and open.

We Built the Chinese Room

What we built

- A conditional probability estimator over token sequences
- Trained on the combinatorially vast space of human-written text
- Using attention to dynamically route context across any distance
- With *no* explicit world model, no causal structure, no truth signal

ChatGPT, Claude, Gemini: billions of parameters, no grounding.

Searle's thought experiment **became a product.**

And it works.

It passes the bar exam.

It diagnoses rare diseases from notes.

It writes code, poetry, legal briefs.

Does it understand any of it?

Searle said: impossible.

We built it anyway.

The question is now empirical — and open.

What Does “Understand” Mean?

Searle's argument turns on a word we haven't defined.

What Does “Understand” Mean?

Searle’s argument turns on a word we haven’t defined.

Behavioral criterion

If a system responds to every situation the way an understander would, what more could “understanding” mean?

“If it walks like a duck. . .”

What Does “Understand” Mean?

Searle’s argument turns on a word we haven’t defined.

Behavioral criterion

If a system responds to every situation the way an understander would, what more could “understanding” mean?

“If it walks like a duck. . .”

Intentional criterion

Understanding requires *aboutness*: mental states genuinely about things in the world. Syntax can never produce semantics.

What Does “Understand” Mean?

Searle’s argument turns on a word we haven’t defined.

Behavioral criterion

If a system responds to every situation the way an understander would, what more could “understanding” mean?

“If it walks like a duck. . .”

Intentional criterion

Understanding requires *aboutness*: mental states genuinely about things in the world. Syntax can never produce semantics.

The empirical question

Do LLMs generalize, reason, and fail in the ways a *understander* would — or a *very good pattern-matcher* would?

What Does “Understand” Mean?

Searle’s argument turns on a word we haven’t defined.

Behavioral criterion

If a system responds to every situation the way an understander would, what more could “understanding” mean?

“If it walks like a duck. . .”

Intentional criterion

Understanding requires *aboutness*: mental states genuinely about things in the world. Syntax can never produce semantics.

The empirical question

Do LLMs generalize, reason, and fail in the ways a *understander* would — or a *very good pattern-matcher* would? **And does it matter?**

Summary

- **Language modeling:** next-word prediction
≡ sequence probability ≡ generation.

Summary

- **Language modeling:** next-word prediction
≡ sequence probability ≡ generation.
- **Why it's hard:** context space exceeds
atoms in the universe; lookup fails; n-grams
miss long dependencies.

Summary

- **Language modeling:** next-word prediction
≡ sequence probability ≡ generation.
- **Why it's hard:** context space exceeds
atoms in the universe; lookup fails; n-grams
miss long dependencies.
- **Transformers:** attention gives
variable-length, long-range, parallel context
— at $\mathcal{O}(T^2)$ cost.

Summary

- **Language modeling:** next-word prediction
≡ sequence probability ≡ generation.
- **Why it's hard:** context space exceeds
atoms in the universe; lookup fails; n-grams
miss long dependencies.
- **Transformers:** attention gives
variable-length, long-range, parallel context
— at $\mathcal{O}(T^2)$ cost.
- **Ecosystem:** HuggingFace, quantization,
LoRA enable local clinical deployment.

- **Language modeling:** next-word prediction
≡ sequence probability ≡ generation.
- **Why it's hard:** context space exceeds
atoms in the universe; lookup fails; n-grams
miss long dependencies.
- **Transformers:** attention gives
variable-length, long-range, parallel context
— at $\mathcal{O}(T^2)$ cost.
- **Ecosystem:** HuggingFace, quantization,
LoRA enable local clinical deployment.
- **Open:** hallucination, decoding, position
generalization, reasoning, agents.

Summary

- **Language modeling:** next-word prediction
≡ sequence probability ≡ generation.
- **Why it's hard:** context space exceeds atoms in the universe; lookup fails; n-grams miss long dependencies.
- **Transformers:** attention gives variable-length, long-range, parallel context — at $\mathcal{O}(T^2)$ cost.
- **Ecosystem:** HuggingFace, quantization, LoRA enable local clinical deployment.
- **Open:** hallucination, decoding, position generalization, reasoning, agents.

Companion notebook

- §1 Context match statistics
- §2 N-gram models & generation
- §3 Transformer from scratch
- §4 Decoding strategies
- §5 HuggingFace inference

Key readings

Vaswani et al. (2017) "Attention is All You Need"
Brown et al. (2020) "GPT-3"
Liu et al. (2023) "Lost in the Middle"
Hu et al. (2021) "LoRA"

The Historical Path: RNNs to Transformers

Why “decoder-only”?

The original Transformer (Vaswani 2017) had an *encoder* (reads the full source) and a *decoder* (generates the target). GPT-family models dropped the encoder — the decoder alone, trained on next-token prediction, turns out to be enough for almost everything.

RNNs and LSTMs (1990s–2017)

Handle variable-length sequences naturally. But computation is **sequential** — each step depends on the last — making parallelization impossible. Gradients also **vanish** over long distances, limiting effective context.

Transformers (Vaswani et al., 2017)

All-to-all attention: every position can directly attend to every other in a single layer. Fully parallel. Scales to billions of parameters on modern hardware.

“Attention is All You Need”

Further Reading: Bidirectional & Encoder–Decoder Architectures

Self-study only — not covered in lecture

ELMo Peters et al. (2018): contextualized embeddings from bidirectional LSTMs. First major contextual representation.

BERT Devlin et al. (2018): bidirectional encoder, Masked Language Modeling + Next Sentence Prediction. Dominated benchmarks 2018–2021.

T5 / BART Encoder–decoder: encoder reads full source, decoder generates target autoregressively. Used in translation and summarization.

GPT lineage Decoder-only autoregressive models (GPT-1 through GPT-4o).
Pre-trained on next-token prediction.

Key reading: Vaswani et al. (2017); Devlin et al. (2018) “BERT”; Brown et al. (2020) “GPT-3”

§5 Notebook: How BioMedLM Tokenizes Medical Terms

Subword tokenization (BPE) splits on morpheme-like boundaries — frequent roots become single tokens:

§5 Notebook: How BioMedLM Tokenizes Medical Terms

Subword tokenization (BPE) splits on morpheme-like boundaries — frequent roots become single tokens:

```
"myocardial infarction"
```

```
→ ['my', 'ocardial', 'infarction']
```

```
"electrocardiogram"
```

```
→ ['electro', 'cardi', 'ogram']
```

```
"hypertension"
```

```
→ ['hyper', 'tension']
```

```
"acetaminophen"
```

```
→ ['acet', 'aminophen']
```

```
"MIMIC-IV"
```

```
→ ['MI', 'MIC', '-', 'IV']
```

```
"hemodynamically unstable"
```

```
→ ['hem', 'od', 'ynamically',  
    'unstable']
```

§5 Notebook: How BioMedLM Tokenizes Medical Terms

Subword tokenization (BPE) splits on morpheme-like boundaries — frequent roots become single tokens:

```
"myocardial infarction"
```

```
→ ['my', 'ocardial', 'infarction']
```

```
"electrocardiogram"
```

```
→ ['electro', 'cardi', 'ogram']
```

```
"hypertension"
```

```
→ ['hyper', 'tension']
```

```
"acetaminophen"
```

```
→ ['acet', 'aminophen']
```

```
"MIMIC-IV"
```

```
→ ['MI', 'MIC', '-', 'IV']
```

```
"hemodynamically unstable"
```

```
→ ['hem', 'od', 'ynamically',  
    'unstable']
```

Roots like cardio, hyper, electro earn their own tokens. Rare terms fragment — token count affects cost and performance.

§5 Notebook: Perplexity — What the Model Has Learned

Perplexity measures how surprised the model is by a sequence. Lower = more expected.

§5 Notebook: Perplexity — What the Model Has Learned

Perplexity measures how surprised the model is by a sequence. Lower = more expected.

Sentence (BioMedLM, trained on PubMed)	PPL
"The patient was admitted with ST-elevation myocardial..."	12.3
"Troponin levels were elevated consistent with acute..."	15.9
"The patient was started on aspirin, heparin..."	16.6
"The quick brown fox jumps over the lazy dog."	107.8
Random gibberish tokens	326
Scrambled clinical words (correct words, wrong order)	11,043

§5 Notebook: Perplexity — What the Model Has Learned

Perplexity measures how surprised the model is by a sequence. Lower = more expected.

Sentence (BioMedLM, trained on PubMed)	PPL
"The patient was admitted with ST-elevation myocardial..."	12.3
"Troponin levels were elevated consistent with acute..."	15.9
"The patient was started on aspirin, heparin..."	16.6
"The quick brown fox jumps over the lazy dog."	107.8
Random gibberish tokens	326
Scrambled clinical words (correct words, wrong order)	11,043

What this tells us

BioMedLM has internalized clinical language patterns so deeply that coherent biomedical prose is $\sim 9\times$ less surprising than ordinary English — and correct word order matters far more than correct vocabulary (scrambled $\approx 1000\times$ worse than gibberish).

Challenge: Indirect Position Dependence

Attention is order-blind without help

Positional encoding is **learned**, not architecturally enforced. The model must figure out how to use it — and doesn't always succeed.

Challenge: Indirect Position Dependence

Attention is order-blind without help

Positional encoding is **learned**, not architecturally enforced. The model must figure out how to use it — and doesn't always succeed.

Length generalization

Models often degrade on sequences **longer** than their training contexts, even with very long nominal context windows.

Challenge: Indirect Position Dependence

Attention is order-blind without help

Positional encoding is **learned**, not architecturally enforced. The model must figure out how to use it — and doesn't always succeed.

Length generalization

Models often degrade on sequences **longer** than their training contexts, even with very long nominal context windows.

Lost in the Middle

Models attend much more strongly to the beginning and end of a long context than to the middle.

Liu et al., 2023 — “Lost in the Middle”

Challenge: Indirect Position Dependence

Attention is order-blind without help

Positional encoding is **learned**, not architecturally enforced. The model must figure out how to use it — and doesn't always succeed.

Length generalization

Models often degrade on sequences **longer** than their training contexts, even with very long nominal context windows.

Lost in the Middle

Models attend much more strongly to the beginning and end of a long context than to the middle.

Liu et al., 2023 — “Lost in the Middle”

Why this matters clinically

A long clinical note may have the most relevant finding buried in the middle — exactly where the model is least likely to attend.